

中间代码生成模块设计文档

作者：71123203刘东平 任务：中间代码生成

1. 模块概述

本模块负责编译器前端的第三阶段——中间代码生成。输入是语法分析阶段产生的语法树，输出三地址语句（四元式），后续可进一步生成 LLVM IR。针对 C 语言子集实现，配合 Lex、Yacc 模块工作。

2. 支持的 C 子集

类别	内容
基本类型	int, float, char, void
声明	局部/全局变量、数组（含变长数组 []）
赋值	=, +=, -=, *=
算术	+ - * / %、括号、类型转换
自增/自减	++ --（前缀/后缀）
关系/逻辑	< > <= >= == != && !
控制流	if-else, while, do-while, for, break, continue
函数	定义、调用、return
数组	一维数组读写
指针	取地址 &、解引用 *
条件表达式	? :
字符串	字符串字面量
逗号表达式	expr1, expr2

3. 核心数据结构

3.1 AST 节点

```
enum class ASTNodeType {
    // 程序结构
    PROGRAM,

    // 声明与定义
    FUNC_DEF,          // 函数定义
    DECL,              // 变量声明
    PARAM_DECL,        // 函数参数声明

    // 语句
    BLOCK,             // 复合语句
    ASSIGN,            // 赋值语句
    EXPR_STMT,         // 表达式语句
    EMPTY_STMT,        // 空语句(;)
    RETURN_STMT,       // return 语句
    BREAK_STMT,        // break 语句
    CONTINUE_STMT,     // continue 语句

    // 控制流
    IF_STMT,           // if-else
    WHILE_STMT,        // while
    DO_WHILE_STMT,     // do-while
    FOR_STMT,          // for

    // 表达式
    BINOP,             // 二元运算
    UNOP,              // 一元运算
    INC_DEC,           // 自增/自减
    CAST,              // 类型转换
    COND_EXPR,         // 条件表达式(?:)
    COMMA_EXPR,        // 逗号表达式

    // 函数调用
    FUNC_CALL,         // 函数调用
    ARG_LIST,          // 实参列表

    // 数组
    ARRAY_ACCESS,      // 数组访问
    ARRAY_INIT,        // 数组初始化

    // 操作数
    IDENT,             // 标识符
    INT_LITERAL,        // 整数
    FLOAT_LITERAL,     // 浮点数
    CHAR_LITERAL,      // 字符
    STRING_LITERAL,    // 字符串
};
```

```
enum class DataType {
    INT, FLOAT, CHAR, VOID,
    POINTER,
    ARRAY,
    UNKNOWN,
};

enum class OpType {
    // 算术
    ADD, SUB, MUL, DIV, MOD,
    // 关系
    LT, LE, GT, GE, EQ, NE,
    // 逻辑
    AND, OR, NOT,
    // 一元
    UMINUS, UPLUS, ADDR, Deref,
    // 自增/自减
    INC, DEC,
    // 其他
    ASSIGN, CAST,
};
```

```
struct ASTNode {
    ASTNodeType      type;
    std::string      lexeme;          // 标识符名或字面量
    OpType           op;              // 操作符类型
    DataType          data_type;      // 语义类型
    int              line_no;
    int              col_no;
    std::vector<ASTNode*> children;

    // 语义属性（遍历时填充）
    std::string      place;           // 结果临时变量
    std::vector<int> true_list;       // 真出口回填链
    std::vector<int> false_list;      // 假出口回填链
    std::vector<int> break_list;      // break 跳转目标
    std::vector<int> continue_list;   // continue 跳转目标
    int              width;           // 类型字节数
    bool             is_lvalue;       // 是否左值

    // 数组相关
    DataType          array_elem_type;
    int               array_size;     // 0 表示变长数组

    // 自增/自减相关
    bool              is_prefix;      // true=前缀 ++i

    // 条件表达式相关
    int               mid_label;      // ? : 中的 ? 跳转目标
};
```

3.2 符号表

采用作用域链结构。

```
struct SymbolEntry {
    std::string name;
    DataType    type;
    int         offset;      // 相对栈帧的偏移量
    int         width;
    bool        is_array;
    int         array_size;
    bool        is_param;
    bool        is_pointer;
    bool        is_const;
    int         line_no;
    bool        initialized;
};

class SymbolTable {
public:
    SymbolTable* parent;
    std::map<std::string, SymbolEntry> entries;
    int         current_offset;
    std::string scope_name;
    std::vector<std::string> strings;

    void        insert(const SymbolEntry& e);
    SymbolEntry* lookup(const std::string& name);
    SymbolTable* enter_scope(const std::string& name);
    SymbolTable* exit_scope();

    bool is_in_loop() const;
    void set_loop_info(int break_label, int continue_label);
    int  get_break_label() const;
    int  get_continue_label() const;

private:
    int    loop_depth_;
    int    break_target_;
    int    continue_target_;
};
```

3.3 四元式

格式：(op, arg1, arg2, result)

```
struct Quadruple {
    std::string op;
    std::string arg1;
    std::string arg2;
    std::string result;
    int         index;
};
```

常用 op：

语句	op	示例
赋值	:=	t1 := x
复合赋值	+= -= *= /=	t := x += y
二元运算	+ - * / %	t3 := t1 + t2
取反	uminus	t2 := -t1
关系运算	< > <= >= == !=	t3 := t1 < t2

逻辑与/或/非	&& !	示例 t1 := t1 && t2
条件跳转	if_true / if_false	if t1 goto L1
无条件跳转	goto	goto L1
标号	label	L1:
函数调用	call	t := call f, n
参数传递	param	param t1
函数入口/退出	func_begin / func_end	func_begin f
返回	return	return t1
数组写/读	[] = []	a[i] := t1 / t1 := a[i]
取地址/解引用	& =*	t1 := &x / t1 := *ptr
类型转换	cast	t2 := (float)t1
自增/自减	++ ++_post	t := ++x / t := x++
全局字符串	str_const	str0 := "hello"

3.4 代码发射器

```
class CodeEmitter {
public:
    int emit(const std::string& op,
             const std::string& arg1,
             const std::string& arg2,
             const std::string& result);

    std::string new_temp();    // 生成 t0, t1, ...
    std::string new_label();  // 生成 L0, L1, ...

    void backpatch(const std::vector<int>& list, const std::string& label);
    std::vector<int> merge(const std::vector<int>& l1,
                          const std::vector<int>& l2);

    const std::vector<Quadruple>& get_code() const;
    void print_code() const;

    void set_break_target(const std::string& label);
    void set_continue_target(const std::string& label);
    std::string get_break_target() const;
    std::string get_continue_target() const;

private:
    std::vector<Quadruple> code_;
    int temp_count_ = 0;
    int label_count_ = 0;
    std::string break_target_;
    std::string continue_target_;
};
```

4. 语义动作设计

4.1 声明

```
D -> T id ;
    { symtab.insert(id.lexeme, T.type, offset);
      offset += T.width; }
```

类型宽度 : int=4, float=8, char=1, pointer=8

4.2 赋值与算术

```
S -> id = E ;
    { emit(':=', E.place, '-', symtab.lookup(id.lexeme).name); }

E -> E1 + E2 { E.place = new_temp();
               emit('+', E1.place, E2.place, E.place); }
    | - E1   { E.place = new_temp();
               emit('uminus', E1.place, '-', E.place); }
    | id     { E.place = symtab.lookup(id.lexeme).name; }
```

4.3 自增/自减

```
// 前缀 ++id
E -> ++ id { E.place = new_temp();
            p = symtab.lookup(id.lexeme);
            emit('++', p.name, '-', E.place); }

// 后缀 id++
E -> id ++ { E.place = new_temp();
            p = symtab.lookup(id.lexeme);
            emit(':=', p.name, '-', E.place);
            emit('++', p.name, '-', p.name); }
```

4.4 if-else (回填)

```
S -> if ( B ) S1 else S2
    { backpatch(B.true_list, next_label());
      emit('goto', '-', '-', '?');
      backpatch(B.false_list, next_label()); }
    { backpatch({goto_idx}, next_label()); }
```

4.5 while

```
S -> while ( B ) S1
    { emit('label', '-', '-', begin);
      backpatch(B.true_list, next_label());
      emit('goto', '-', '-', begin);
      backpatch(B.false_list, next_label()); }
```

4.6 短路求值

```

// A && B
// A 为假则整体为假，无需计算 B
B -> B1 && B2
    { backpatch(B1.true_list, next_label());
      B.true_list = B2.true_list;
      B.false_list = merge(B1.false_list, B2.false_list); }

// A || B
// A 为真则整体为真，无需计算 B
B -> B1 || B2
    { backpatch(B1.false_list, next_label());
      B.true_list = merge(B1.true_list, B2.true_list);
      B.false_list = B2.false_list; }

```

4.7 break / continue

```

S -> break ;
    { emit('goto', '-', '-', symtab.get_break_target()); }

S -> continue ;
    { emit('goto', '-', '-', symtab.get_continue_target()); }

```

4.8 条件表达式

```

E -> B ? E1 : E2
    { emit('if_false', B.place, '-', mid);
      emit(':=' , E1.place, '-', t);
      emit('goto', '-', '-', end);
      emit('label', '-', '-', mid);
      emit(':=' , E2.place, '-', t);
      emit('label', '-', '-', end);
      E.place = t; }

```

4.9 数组

```

// 读取
E -> id [ E1 ]
    { emit('[' , E2.place, E1.place, symtab.lookup(id.lexeme).name); }

// 写入
S -> id [ E1 ] = E2 ;
    { emit('[' , E2.place, E1.place, symtab.lookup(id.lexeme).name); }

```

4.10 函数调用

```

S -> id ( ArgList ) ;
    { for arg in reverse(ArgList):
      emit('param', arg.place, '-', '-');
      t = new_temp();
      emit('call', id.lexeme, ArgList.size, t); }

```

5. AST 遍历框架

```

visit(node):
    switch node.type:
        PROGRAM:
            symtab = new SymbolTable()
            for child in node.children: visit(child)

        FUNC_DEF:
            emit('func_begin', node.lexeme, '-', '-')
            symtab.enter_scope(node.lexeme)
            // 处理参数...
            for child in node.children: visit(child)
            emit('func_end', node.lexeme, '-', '-')
            symtab.exit_scope()

        BLOCK:
            symtab.enter_scope("block")
            for child in node.children: visit(child)
            symtab.exit_scope()

        DECL:      visit_decl(node)
        ASSIGN:    visit_assign(node)
        IF_STMT:   visit_if(node)
        WHILE_STMT: visit_while(node)
        DO_WHILE_STMT: visit_do_while(node)
        FOR_STMT:  visit_for(node)
        BREAK_STMT: visit_break(node)
        CONTINUE_STMT: visit_continue(node)
        BINOP:     visit_binop(node)
        UNOP:      visit_unop(node)
        INC_DEC:   visit_inc_dec(node)
        ARRAY_ACCESS: visit_array_access(node)
        FUNC_CALL: visit_func_call(node)
        COND_EXPR: visit_cond_expr(node)
        CAST:      visit_cast(node)

```

6. 回填机制

```

// 生成跳转指令，目标用 '?' 占位
int idx = emit('if_true', place, '-', '?');
true_list = { idx };

// 目标确定后回填
backpatch(list, label):
    for i in list:
        code[i].result = label

// 回填链合并
merge(l1, l2):
    return l1.concat(l2)

```

7. 多层循环中的 break/continue

符号表维护循环栈：


```
class SymbolTable {
    std::vector<std::pair<int, int>> loop_stack_;

    void enter_loop(int break_label, int continue_label) {
        loop_stack_.push_back({break_label, continue_label});
    }

    void exit_loop() {
        loop_stack_.pop_back();
    }
};
```

8. 错误处理

语义错误类型：

错误	说明
E001	未声明的标识符
E002	重复声明
E003	类型不匹配
E004	左值不可赋值
E005	函数参数类型不匹配
E006	函数参数个数不匹配
E007	break/continue 不在循环中
E008	void 变量声明
E009	void 函数返回值类型错误
E010	void 表达式用于值上下文

```
class SemanticError {
public:
    enum class ErrorType {
        UNDECLARED, REDEFINED, TYPE_MISMATCH, NOT_LVALUE,
        PARAM_TYPE_MISMATCH, PARAM_COUNT_MISMATCH,
        BREAK_OUTSIDE_LOOP, CONTINUE_OUTSIDE_LOOP,
        VOID_VARIABLE, VOID_RETURN_MISMATCH, VOID_EXPRESSION_VALUE,
    };

    SemanticError(ErrorType type, const std::string& msg,
                  int line, int col);

    void report() const;
};

class ErrorHandler {
    std::vector<SemanticError> errors_;
    bool has_error_ = false;

public:
    void add_error(SemanticError&& err);
    bool has_error() const { return has_error_; }
    void report_all() const;
};
```

9. 模块接口

方向	内容	格式
输入	语法树根节点	ASTNode*
输出	四元式序列	vector<Quadruple>
输出	错误列表	vector<SemanticError>

10. Token 映射

Lex Token	AST 节点	四元式 op
ADD SUB MUL DIV MOD	BINOP	+ - * / %
LT LE GT GE EQ NE	BINOP	< <= > >= == !=
AND OR NOT	BINOP / UNOP	&& !
INC DEC	INC_DEC	++ --
ASSIGN	ASSIGN	:=
& *	UNOP	& *